Économétrie des Séries Temporelles

Fiche TD R #2

Processus ARMA stationnaires

Packages

```
library(readr)
library(zoo)
library(astsa)
library(forecast)
library(stats)
library(tseries)
library(aTSA)
library(lmtest)
#install.packages("aTSA")
```

Données (identiques au TP1)

Nice : https://github.com/bilelsanhaji/EdSTM1/blob/main/Data/SH_MIN006088001.csv

Paris : https://github.com/bilelsanhaji/EdSTM1/blob/main/Data/SH_MIN175114001.csv

```
urlSHnice <- "https://raw.githubusercontent.com/bilelsanhaji/EdSTM1/refs/heads/main/Data/SH_MIN00608800
InsoNice <- read_delim(urlSHnice,
                       delim = ";",
                       escape_double = FALSE,
                       col_types = cols(YYYYMM = col_date(format = "%Y%m")),
                       comment = "#", trim_ws = TRUE)

urlSHparis <- "https://raw.githubusercontent.com/bilelsanhaji/EdSTM1/refs/heads/main/Data/SH_MIN1751140
InsoParis <- read_delim(urlSHparis,
                        delim = ";",
                        escape_double = FALSE,
                        col_types = cols(YYYYMM = col_date(format = "%Y%m")),
                        comment = "#", trim_ws = TRUE)
```

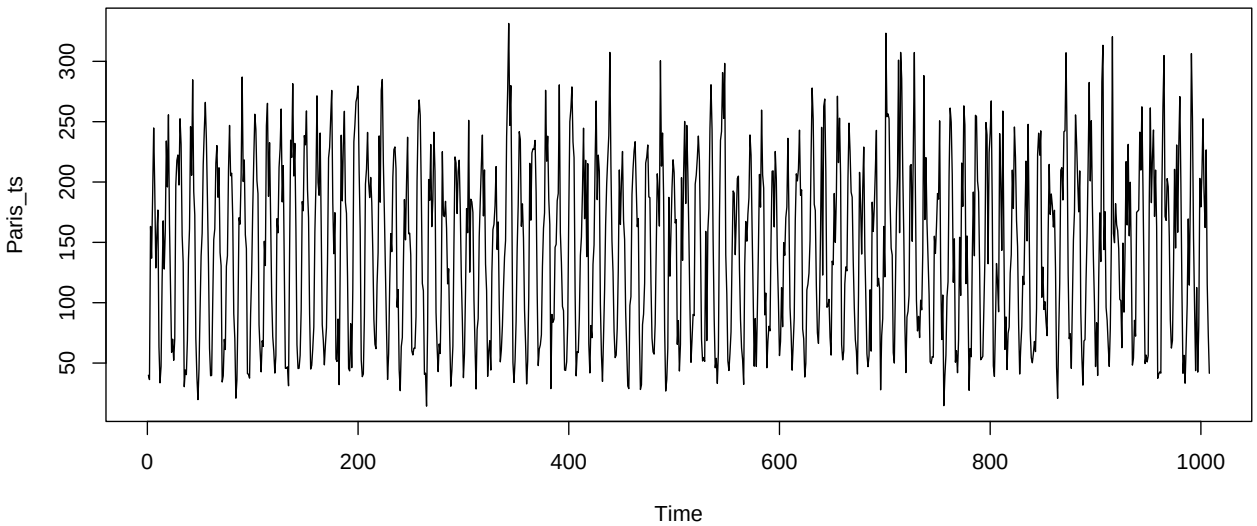
```
Nice_ts = ts(InsoNice$VALEUR)
Paris_ts = ts(InsoParis$VALEUR)
```

Exercice 1

À partir des données d'insolation de Nice et Paris, utilisez les séries pour

(a) donner une représentation graphique et tester statistiquement :

```r
plot(Nice_ts)
plot(Paris_ts)
```



1. la stationarité

#KPSS

```r
# tseries::kpss.test()
#x = rnorm(1000)  # is level stationary
#tseries::kpss.test(x)
tseries::kpss.test(Nice_ts)
```

```
##
##  KPSS Test for Level Stationarity
##
## data:  Nice_ts
## KPSS Level = 0.058965, Truncation lag parameter = 7, p-value = 0.1
```

```r
aTSA::kpss.test(Nice_ts)
```

```
## KPSS Unit Root Test
## alternative: nonstationary
##
## Type 1: no drift no trend
##  lag stat p.value
##    7  5.8    0.01
## -----
##  Type 2: with drift no trend
##  lag   stat p.value
##    7 0.0145     0.1
## -----
##  Type 1: with drift and trend
##  lag   stat p.value
##    7 0.0151     0.1
## -----------
## Note: p.value = 0.01 means p.value <= 0.01
##     : p.value = 0.10 means p.value >= 0.10
```

```r
#ts.plot(x)
#y = cumsum(x)  # has unit root
#tseries::kpss.test(y)
#ts.plot(y)
# aTSA::kpss.test()
```

#Dickey-Fuller (ADF)

```r
tseries::adf.test(Nice_ts)
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  Nice_ts
## Dickey-Fuller = -9.551, Lag order = 10, p-value = 0.01
## alternative hypothesis: stationary
```

```r
aTSA::adf.test(Nice_ts)
```

```
## Augmented Dickey-Fuller Test
## alternative: stationary
##
## Type 1: no drift no trend
##      lag   ADF p.value
## [1,]   0 -3.71  0.0100
## [2,]   1 -4.18  0.0100
## [3,]   2 -4.79  0.0100
## [4,]   3 -4.52  0.0100
## [5,]   4 -3.62  0.0100
## [6,]   5 -2.72  0.0100
## [7,]   6 -2.04  0.0418
## Type 2: with drift no trend
##      lag   ADF p.value
## [1,]   0 -12.6    0.01
## [2,]   1 -15.9    0.01
## [3,]   2 -21.9    0.01
## [4,]   3 -26.1    0.01
## [5,]   4 -27.4    0.01
## [6,]   5 -27.1    0.01
## [7,]   6 -25.8    0.01
## Type 3: with drift and trend
##      lag   ADF p.value
## [1,]   0 -12.6    0.01
## [2,]   1 -15.9    0.01
## [3,]   2 -21.9    0.01
## [4,]   3 -26.1    0.01
## [5,]   4 -27.4    0.01
## [6,]   5 -27.2    0.01
## [7,]   6 -25.9    0.01
## ----
## Note: in fact, p.value = 0.01 means p.value <= 0.01
```

#Philips Perron (PP)

```r
tseries::pp.test(Nice_ts)
```

```
##
##  Phillips-Perron Unit Root Test
##
## data:  Nice_ts
## Dickey-Fuller Z(alpha) = -217.81, Truncation lag parameter = 7, p-value
## = 0.01
## alternative hypothesis: stationary
```

```r
stats::PP.test(Nice_ts)
```

```
##
##  Phillips-Perron Unit Root Test
##
## data:  Nice_ts
## Dickey-Fuller = -11.454, Truncation lag parameter = 7, p-value = 0.01
```

```r
aTSA::pp.test(Nice_ts)
```

```
## Phillips-Perron Unit Root Test
## alternative: stationary
##
## Type 1: no drift no trend
##  lag Z_rho p.value
##    7   -20    0.01
## -----
##  Type 2: with drift no trend
##  lag Z_rho p.value
##    7  -218    0.01
## -----
##  Type 3: with drift and trend
##  lag Z_rho p.value
##    7  -218    0.01
## ---------------
## Note: p-value = 0.01 means p.value <= 0.01
```

2. l'autocorrélation

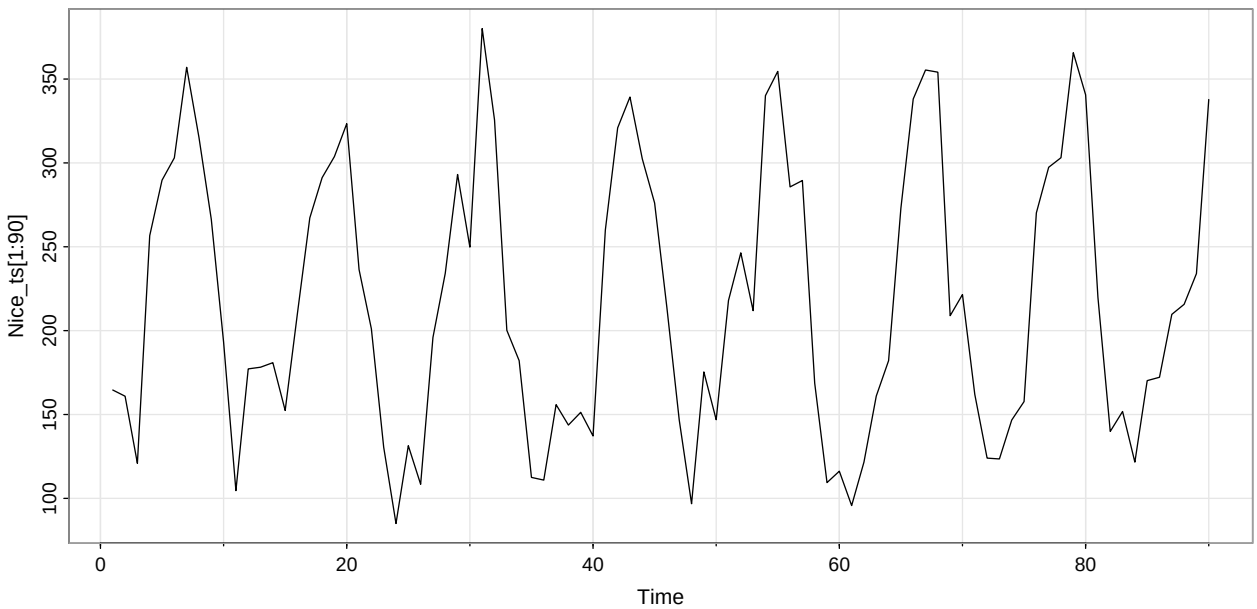
#Durbin-Watson #Breusch-Godfrey #Box-Pierce #Ljung-Box

```r
?Box.test
Box.test(x = Nice_ts, lag = 6, type = "Ljung")
```

```
##
##  Box-Ljung test
##
## data:  Nice_ts
## X-squared = 1910.9, df = 6, p-value < 2.2e-16
```

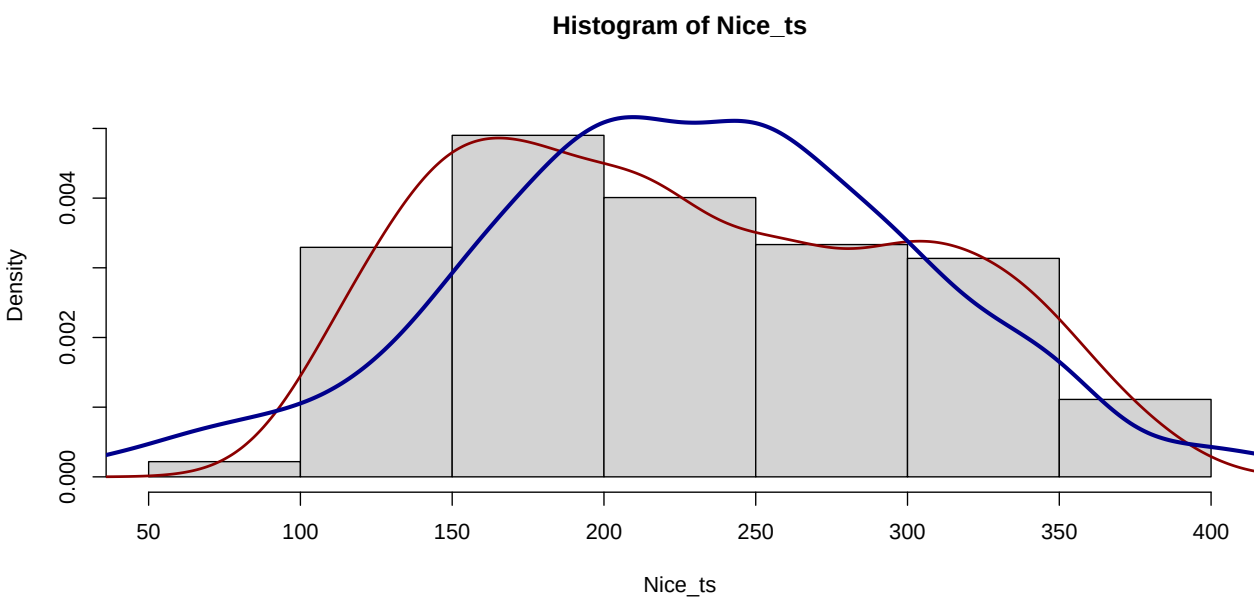
```r
# x <- rnorm (100)
# Box.test (x, lag = 1)
# Box.test (x, lag = 1, type = "Ljung")
```

```r
acf(Nice_ts)
tsplot(Nice_ts[1:90])
```

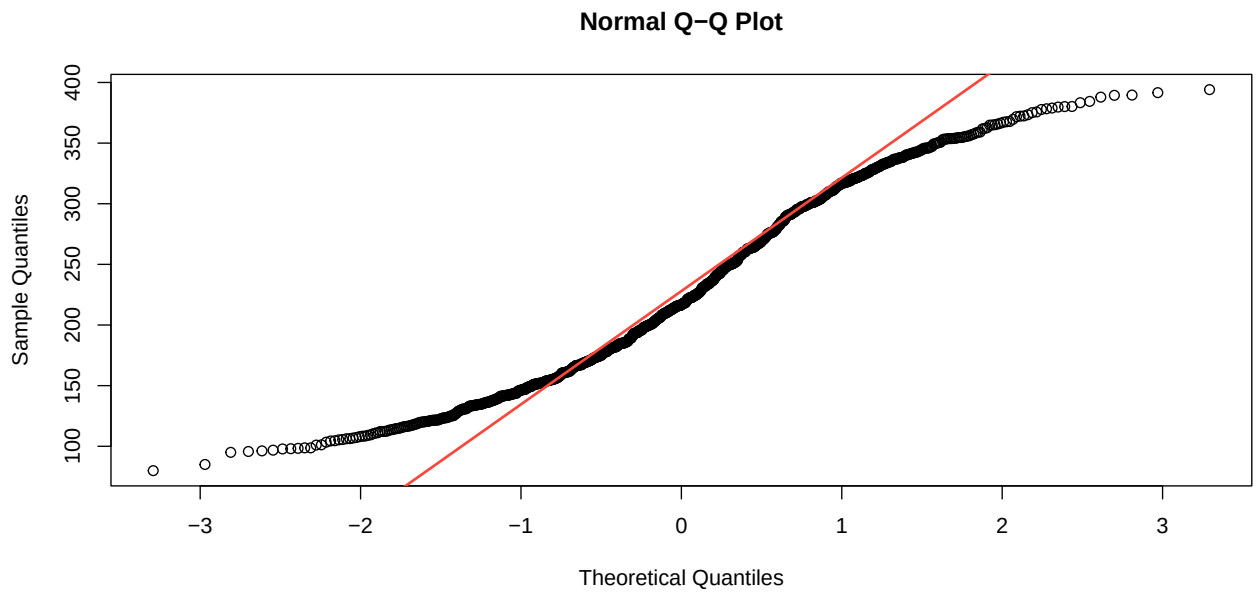


3. la normalité

```
hist(Nice_ts, freq = F, ylim = c(0,0.0055))
lines(density(Nice_ts), col="darkred", lwd=2 )
lines(density(rnorm(1008, mean = mean(Nice_ts), sd = sd(Nice_ts))), col="darkblue", lwd=3 )
```




```
qqnorm(Nice_ts)
qqline(Nice_ts, col=2, lwd=2)
```



#Jarque-Bera

```
jarque.bera.test(Nice_ts)
```

```
##
##  Jarque Bera Test
##
## data:  Nice_ts
## X-squared = 53.152, df = 2, p-value = 2.872e-12
```

```
jarque.bera.test(rnorm(1000))
```

```
##
##  Jarque Bera Test
##
## data:  rnorm(1000)
## X-squared = 2.5815, df = 2, p-value = 0.2751
```

#Kolmogorov-Smirnov #Shapiro-Wilk

(b) estimer et interpréter un AR(1) pour chaque série, puis, sur les résidus :

```
estim.nice = arima(Nice_ts, order = c(1,0,0))
summary(estim.nice)
```

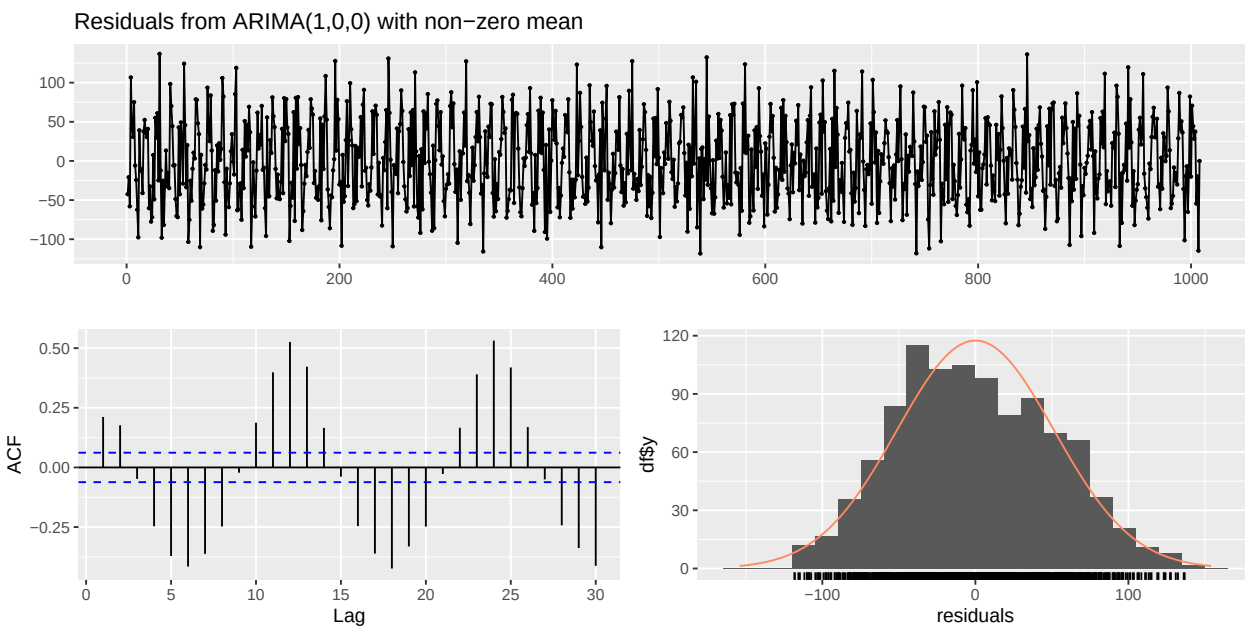
```
##
## Call:
## arima(x = Nice_ts, order = c(1, 0, 0))
##
## Coefficients:
##          ar1  intercept
##       0.7249   226.3135
## s.e.  0.0217     5.8618
##
## sigma^2 estimated as 2634:  log likelihood = -5400.34,  aic = 10806.68
##
## Training set error measures:
##                      ME     RMSE      MAE       MPE     MAPE      MASE
```

```
## Training set 0.04839984 51.32466 42.49581 -5.990051 21.06189 0.9444972
##                    ACF1
## Training set 0.2115974
```

```r
coeftest(estim.nice)
```

```
##
## z test of coefficients:
##
##             Estimate Std. Error z value  Pr(>|z|)
## ar1         0.724934   0.021667  33.457 < 2.2e-16 ***
## intercept 226.313527   5.861831  38.608 < 2.2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
checkresiduals(estim.nice)
```




```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(1,0,0) with non-zero mean
## Q* = 688.45, df = 9, p-value < 2.2e-16
##
## Model df: 1.   Total lags used: 10
```

1. tester autocorrelation, normalité et hétéroscédasticité
2. interprétez tous les résultats obtenus
3. discutez la différence qu'il y a entre les séries

Exercice 2

Simulez un processus AR(1) stationnaire avec 50 observations. Puis

(a) "testez" graphiquement et testez statistiquement :

1. la stationarité
2. l'autocorrélation

3. la normalité

(b) estimez la série simulée et discutez les résultats

(c) reproduire les étapes (a) et (b) avec 5000 observations

1. la stationarité
2. l'autocorrélation
3. la normalité

(b) estimez la série simulée et discutez les résultats